

HUO BOXS · AI & Data Portfolio



AI 应用开发 · AI Agent / RAG · 深度学习 · 数据分析

通过可运行项目展示我在 AI 应用工程、LLM/RAG、数据分析与机器学习 方向的实践能力。

About Me

我是 信计专业本科生，主要关注 AI 应用开发、AI Agent / RAG、深度学习与数据分析。

我的项目实践主要围绕三个问题展开：

- 如何把 LLM 真正做成可使用的产品？
- 如何让 AI 系统不仅“能跑”，而且可评测、可解释、可维护？
- 如何通过数据分析与机器学习，从数据中得到可验证的结论？

目前的项目组合：

方向	项目	核心能力
AI 应用 / RAG	Enterprise RAG	Hybrid Retrieval · Rerank · FastAPI · Ollama · Docker · Evaluation
AI Product	FinPilot	LLM Structured Output · Rule Engine · FastAPI · Taro · SQLite
数据分析	电商经营数据分析	Python · SQL · Power BI · RFM · 用户/经营分析
深度学习 / AI 算法	医疗 AI 细胞分类	CNN / ViT · 多模型实验 · 特征分析 · 模型评估

Project 01 | 电商经营数据分析平台

简要概述

定位：数据分析 / 商业分析 / BI 可视化

围绕电商经营数据搭建从 **数据处理** → **指标体系** → **用户分析** → **可视化 Dashboard** → **经营洞察** 的完整分析流程。

项目重点不是单纯制作图表，而是将业务问题拆解成可以量化的指标，并通过用户分层与经营指标辅助定位问题。

分析链路

- 1 原始经营数据
- 2 ↓
- 3 数据清洗 / 字段处理
- 4 ↓
- 5 指标体系构建
- 6 ↓
- 7 经营概览 Overview
- 8 ↓
- 9 用户行为分析
- 10 ↓
- 11 RFM 用户分层
- 12 ↓
- 13 Power BI Dashboard
- 14 ↓
- 15 经营问题与分析结论

Dashboard

以 Overview 页面作为经营分析入口，将核心经营指标集中展示，并通过趋势、结构和用户维度进行下钻。

RFM 用户分析

通过 **Recency / Frequency / Monetary** 对用户进行分层，从“用户总量”进一步进入“用户价值结构”。

分析思路

维度	含义	分析问题
R · Recency	最近一次消费时间	用户最近是否仍然活跃？
F · Frequency	消费频次	用户是否具有稳定消费行为？
M · Monetary	消费金额	用户贡献的价值有多高？

在此基础上将用户划分为不同价值层级，用于支持 **高价值用户识别**、**沉睡用户识别**、**用户运营与复购分析**。

核心能力

- **数据处理**：从原始业务数据中完成清洗、字段处理与分析数据集构建。
- **指标设计**：围绕经营、用户、交易等维度建立指标体系。
- **用户分析**：使用 RFM 对用户进行价值分层。
- **BI 可视化**：使用 Power BI 构建 Overview、用户分析等 Dashboard。
- **业务思维**：从“展示数据”进一步转向“发现问题 → 定位原因 → 支持决策”。

项目价值

这个项目主要体现我将 **数据** → **指标** → **可视化** → **业务问题** 串联起来的能力，而不是只停留在 Power BI 图表制作层面。

Project 02 | FinPilot · AI 零基础预算 Copilot

简要概述

定位：AI Product / LLM 应用开发

FinPilot 是一个基于微信小程序的 **AI 零基础预算 Copilot**。

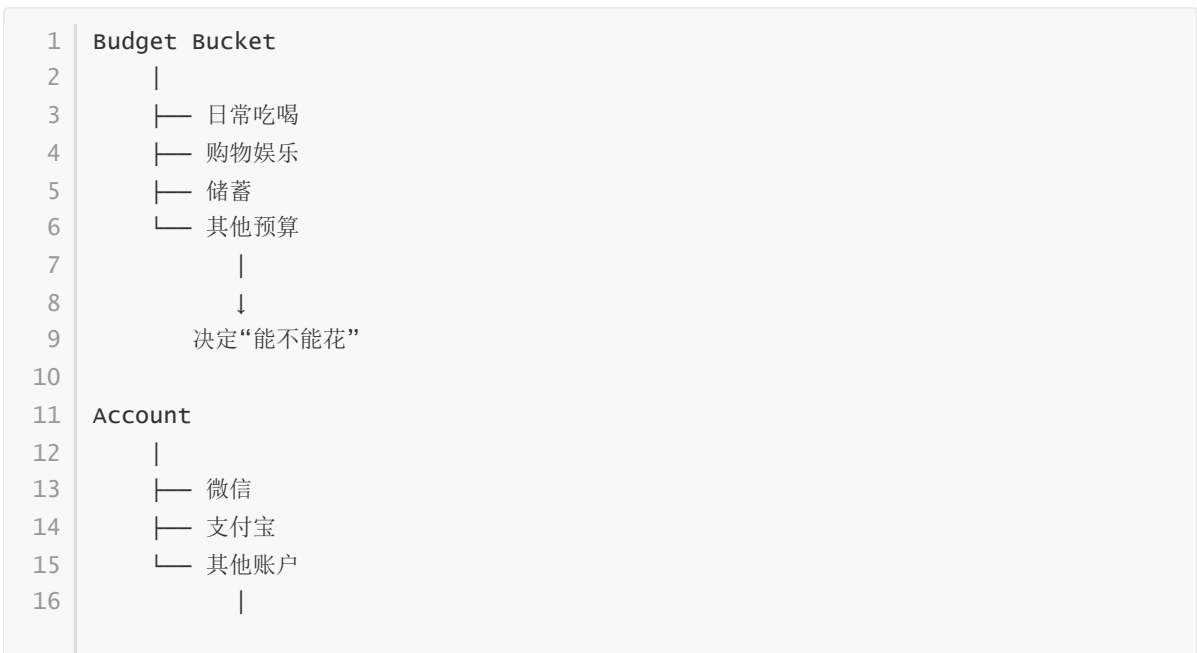
核心思路不是再做一个“记录流水”的记账工具，而是将收入提前划分到不同 **Budget Bucket**，并在消费发生时判断对应预算是否允许支出。

收入 100% 分入预算仓 → 自然语言记录消费 → 规则引擎计算额度 → AI 负责解析与解释 → 用户确认后入账

项目 GitHub: <https://github.com/SomeH-Bosx/FinPilot>

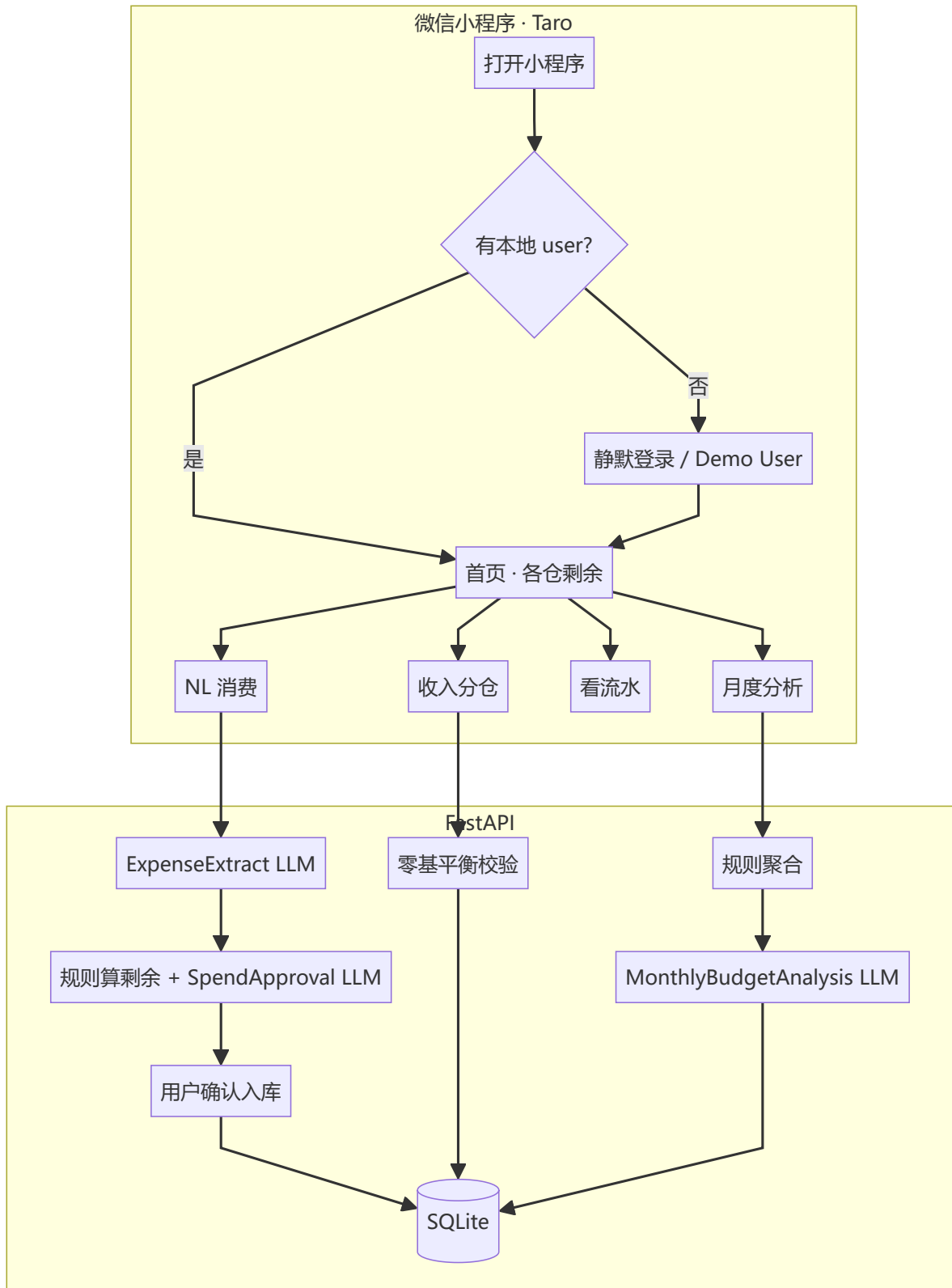
产品模型

FinPilot 将两个容易被混淆的概念拆开：



这样可以避免把“支付账户”直接等同于“预算分类”。

系统架构



AI 与规则的边界

FinPilot 没有让 LLM 直接决定金额和审批结果，而是将系统拆成：

模块	负责内容
LLM	自然语言消费解析、审批理由、月度分析文案
Rule Engine	预算剩余、额度计算、 <code>approve</code> / <code>warn</code> / <code>reject</code>
Pydantic	Structured Output 数据约束
Database	用户、预算仓、账户、流水等状态持久化
Fallback	LLM 不可用时使用 heuristic / 模板兜底

核心原则：

让模型负责理解和表达，让代码负责数字和状态。

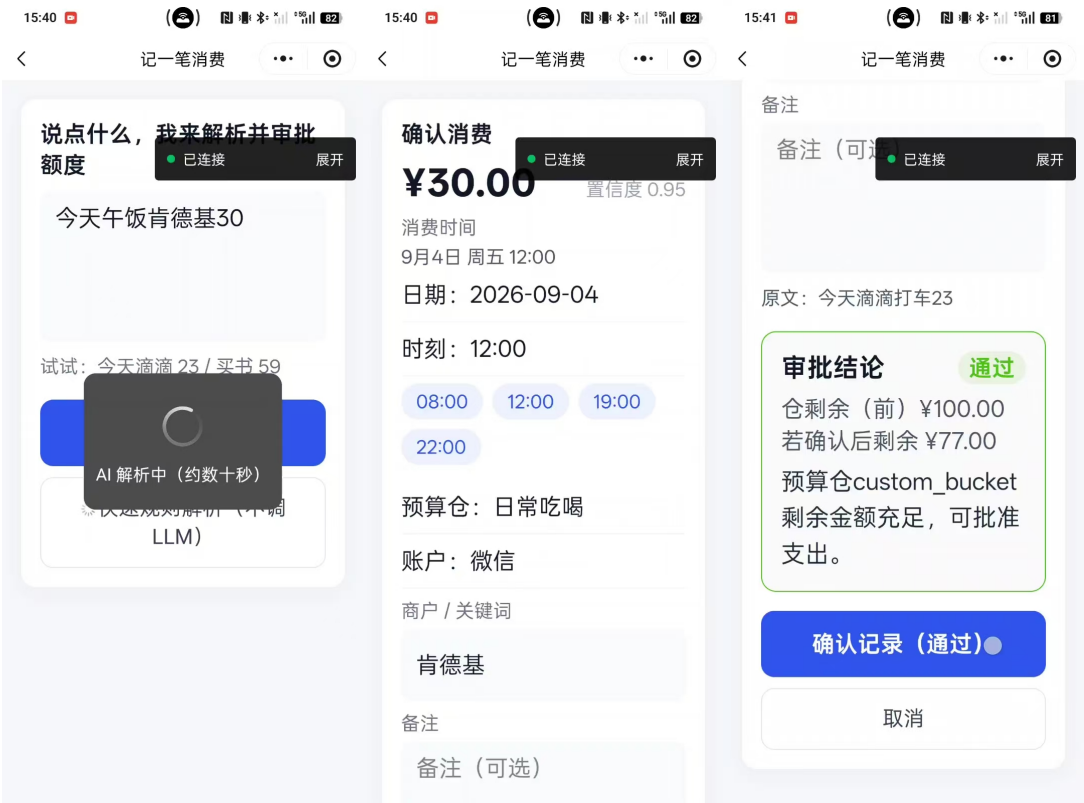
这样可以降低 LLM 幻觉直接修改财务数据的风险。

Demo

初始状态



正常使用



核心亮点

01 · 领域模型先行

拆分 **Budget Bucket / Account**，明确“预算权限”和“支付渠道”的不同职责。

02 · AI 边界清晰

AI 只承担三类任务：

- Natural Language → Structured Data
- 消费审批理由生成
- 月度分析文案

数字计算与审批状态由规则引擎负责。

03 · 零基预算 + 强制储蓄

收入必须 100% 分仓，可支配金额受到预算与储蓄规则约束。

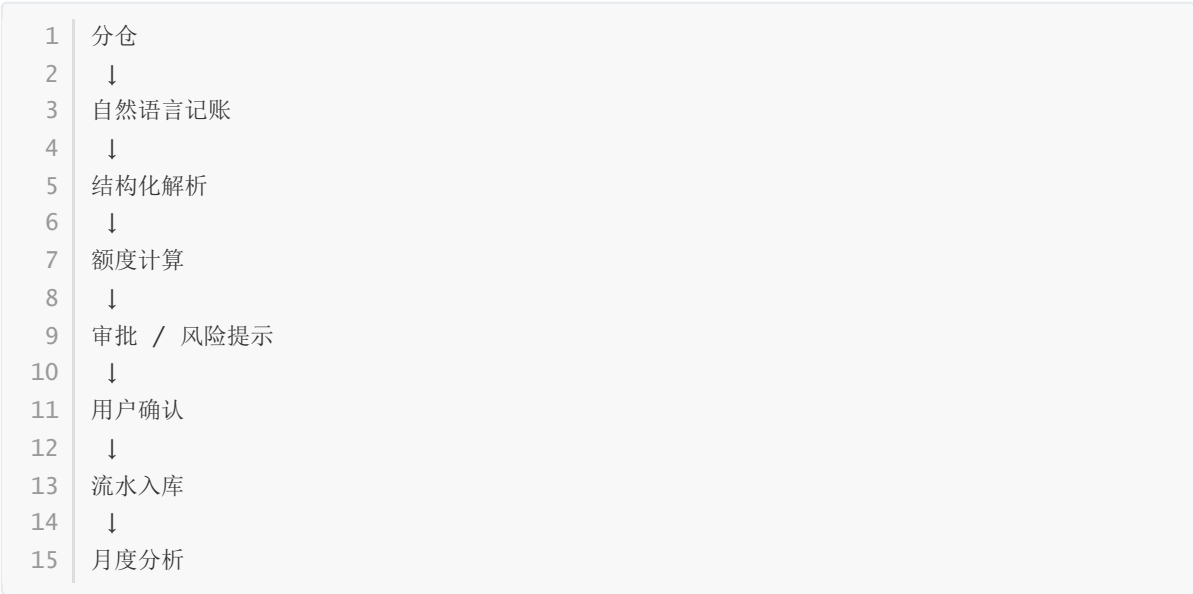
04 · LLM Provider 可切换

通过 `LLMProvider` Adapter 支持：

- Ollama
- 通义
- DeepSeek

同时使用 **Structured Output + Pydantic** 约束模型输出。

05 · 端到端产品闭环



核心难点

难点	解决方案
防止 LLM 修改金额与审批结果	status / 仓剩余 / 零基平衡全部由代码计算
本地 LLM 响应慢	规则与模板优先，云端模型作为可切换方案
“昨天晚饭”时间解析	相对日期 + 餐次规则 + <code>reference_now</code>
储蓄与消费冲突	储蓄仓锁定 + 可支配池约束
真机登录与联调	<code>wx.login</code> → <code>Bearer</code> 鉴权流程

Project 03 | Enterprise RAG · 企业文档知识库问答系统

简要概述

定位：RAG / AI 应用开发 / Agent 基础能力

基于 **LangChain + Chroma + Ollama + FastAPI + Streamlit + Docker** 搭建本地优先的企业文档问答系统。

项目 GitHub: <https://github.com/SomeH-Bosx/enterprise-rag>

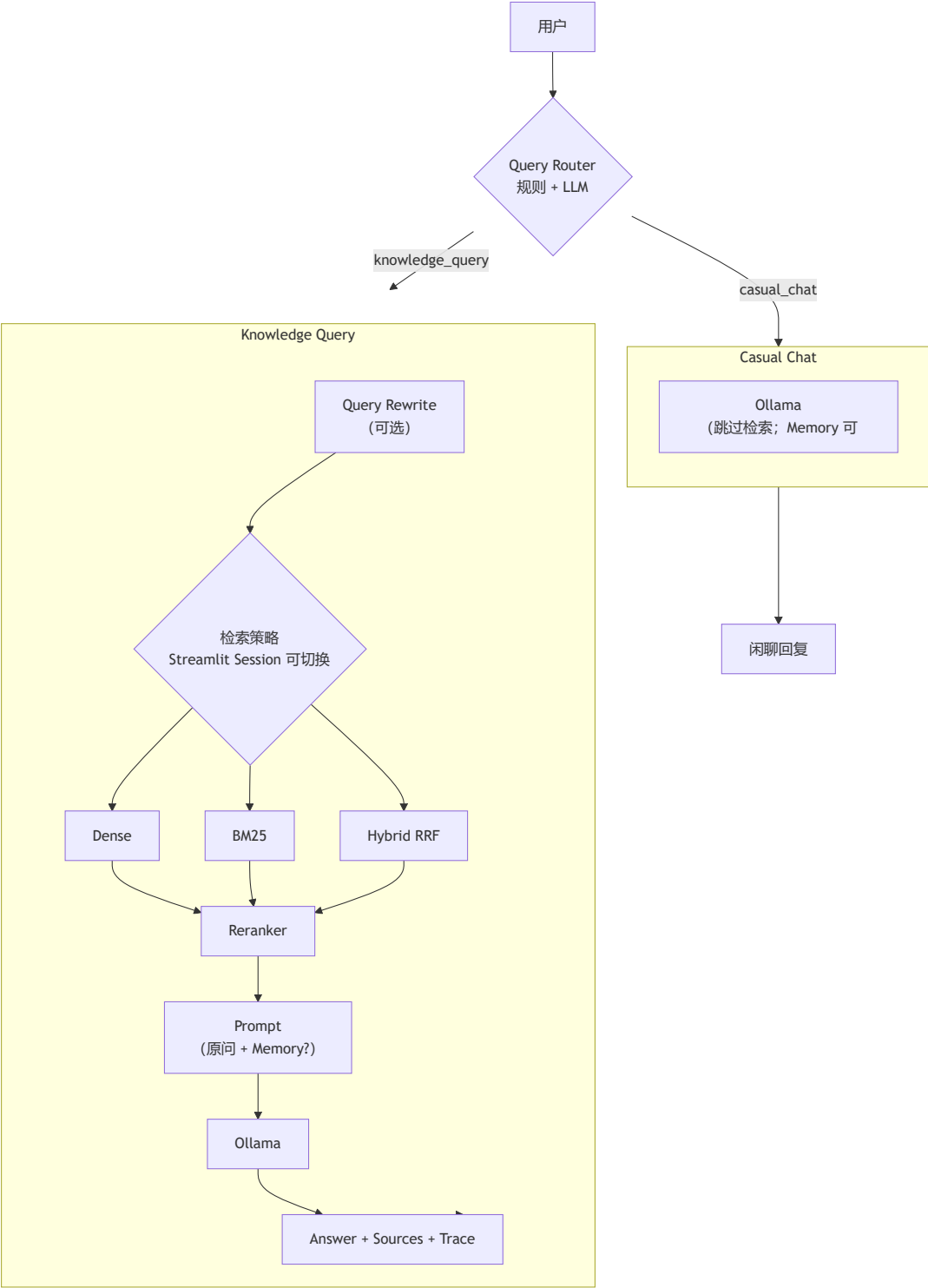
系统支持：

多格式文档入库 → Hybrid Retrieval → Rerank → LLM Generation → Sources / Trace

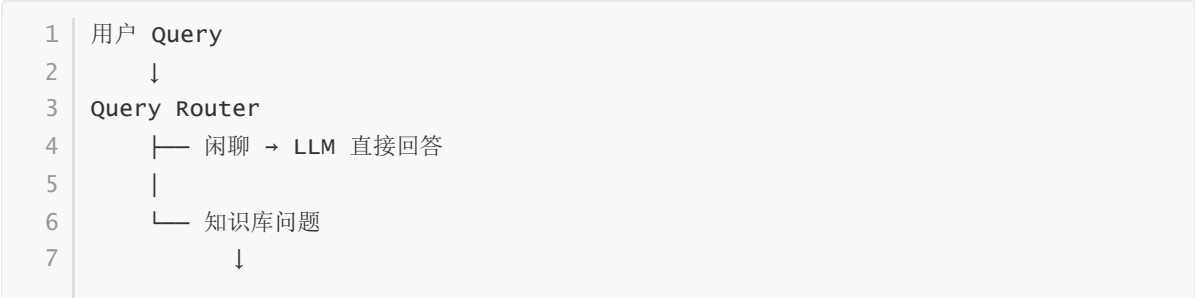
技术栈：

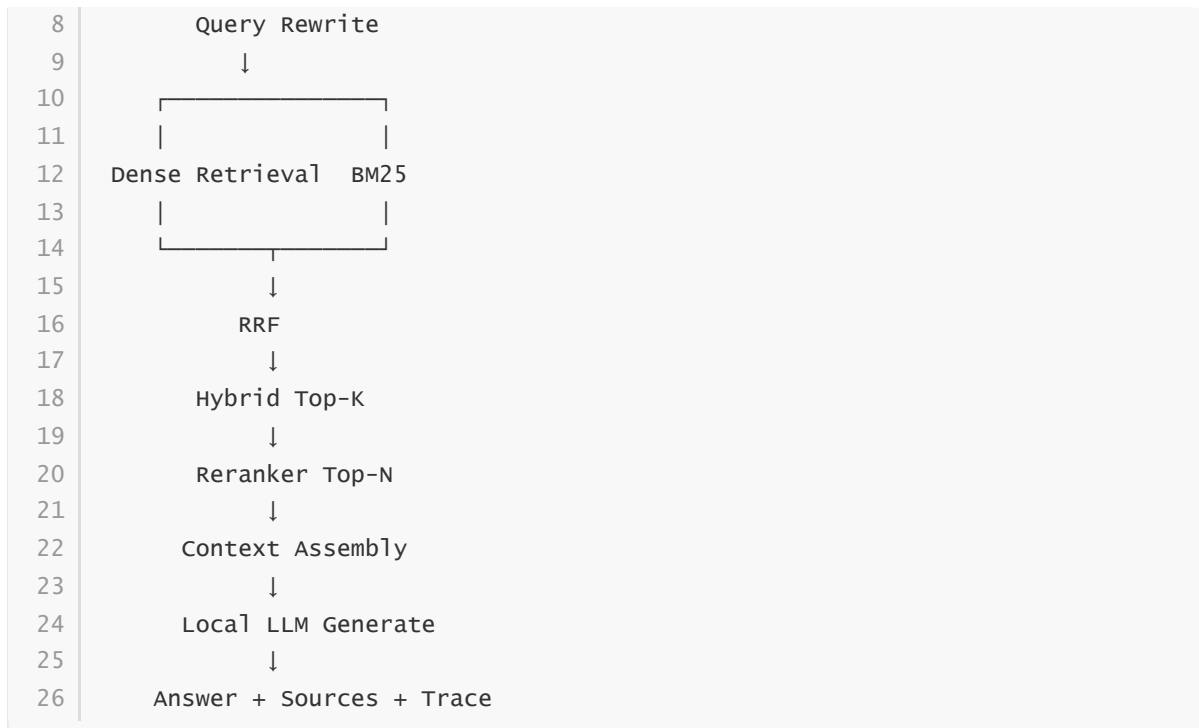
LangChain · Chroma · BM25 · RRF · DashScope Rerank · Ollama · FastAPI · Streamlit · Docker

系统架构



检索流程





评测：从“能跑”到“有证据”

项目没有只展示 Demo，而是通过公平消融实验验证检索策略。

早期的 Rerank 对比存在一个问题：**候选宽度不同会混入实验变量。**

因此将实验改为：

同一批 Hybrid@20 候选 → A: RRF Top-5 → B: Reranker Top-5

最终得到：

配置	Recall@5	Strict Citation (filename, page)
Hybrid@10 / 20 / 30	83.33%	62.07%
Hybrid@20 + Reranker	86.67%	75.86%

实验结果表明，在当前测试集上，单纯扩大 Hybrid 候选数量并没有继续提升 Top-5，而 Reranker 对最终排序质量带来了提升。

Enterprise RAG · Phase5 Evaluation

来源 `evaluation/phase5/candidate_k` · 30 题

CONFIGURATION	RECALL@5	STRICT CITATION
Hybrid@10	83.33%	62.07%
Hybrid@20	83.33%	62.07%
Hybrid@30	83.33%	62.07%
Hybrid@20 + Reranker	86.67%	75.86%

30 题离线召回 (Top-5 计分) ; Strict Citation = (filename, page) 同时匹配。Hybrid@10/20/30 召回质量相同；加入 Reranker 后 Recall@5 83.33%→86.67%，严格引用 62.07%→75.86%。默认服务：Hybrid@20 + Reranker(Top-5)。详见 `docs/eval.md`。

Answer Trace

系统对:

1	Router
2	↓
3	Rewrite
4	↓
5	Retrieval
6	↓
7	Rerank
8	↓
9	Generation

进行链路记录，并返回 Sources 与置信度分解。

回答轨迹

查询分析

问题类型: knowledge_query
路由方式: rules
原问: According to acme_employee_handbook.pdf, how many annual leave days do ACME full-time employees get?
改写问: acme full-time employees annual leave days according to acme_employee_handbook.pdf
改写方法: llam
检索模式: Hybrid (Dense+BM25)
Hybrid (BM25): 并

检索

检索器: Hybrid (Dense + BM25 RRF)
Top-K = 20 · candidates=20
切块可能在句中截断（按字符分割）；块间 overlap 可减轻上下文丢失。

#1 acme_employee_handbook.pdf

> · page=1 · chunk conf 82%

#2 offer先生春招求职一本通

> PDF.pdf · page=76 · chunk conf 51%

#3 迪士尼乐园——东京迪士尼解

> 桥-中英文版.pptx · page=24 · chunk conf 48%

#4 offer先生春招求职一本通

> PDF.pdf · page=85 · chunk conf 51%

#5 迪士尼乐园——东京迪士尼解

> 桥-中英文版.pptx · page=17 · chunk conf 48%

#6 附件7：广东技术师范大学学生手册（2023年3月版）.pdf

> 手册（2023年3月版）.pdf · page=29 · chunk conf 50%

#7 迪士尼乐园——东京迪士尼解

> 桥-中英文版.pptx · page=30 · chunk conf 47%

#8 offer先生春招求职一本通

> PDF.pdf · page=78 · chunk conf 50%

重排

重排器: DashScope Reranker
选用: Top5

#1 acme_employee_handbook.pdf

> · page=1 · chunk conf 87%

#2 附件7：广东技术师范大学学生手册（2023年3月版）.pdf

> 手册（2023年3月版）.pdf · page=29 · chunk conf 29%

#3 附件7：广东技术师范大学学生手册（2023年3月版）.pdf

> 手册（2023年3月版）.pdf · page=33 · chunk conf 29%

#4 附件7：广东技术师范大学学生手册（2023年3月版）.pdf

> 手册（2023年3月版）.pdf · page=31 · chunk conf 29%

#5 附件7：广东技术师范大学学生手册（2023年3月版）.pdf

> 手册（2023年3月版）.pdf · page=34 · chunk conf 29%

生成

LLM: Ollama:qwen2.5:7b
Response Generated

置信度

82%
等级: High
retrieval=0.8342 · rerank=0.7512 ·
source_coverage=1.0 · grounding=0.92
$$0.25 \times 0.834(\text{retrieval_cal}) + 0.5 \times 0.751(\text{rerank}) + 0.1 \times 1.000(\text{source}) + 0.15 \times 0.920(\text{grounding}) = 0.822$$

> 原始 Trace（安全字段）

这使系统能够从“黑盒问答”变成可观察的 RAG Pipeline。

核心亮点

01 · Hybrid Retrieval

同时使用：

- Dense Retrieval
- BM25

- RRF Fusion

通过 Session 配置支持不同检索模式。

02 · Reranker

在召回候选集上进行二阶段排序，而不是盲目扩大召回数量。

03 · Query Router + Query Rewrite

将 知识库问答 与 闲聊 分流；Rewrite 只作用于检索侧，最终生成仍使用原始问题。

04 · Conversation Memory

按 `conversation_id` 持久化消息，并通过有限历史窗口控制 Prompt 长度。

05 · 多格式文档入库

支持：

`pdf / doc / docx / ppt / pptx / md / txt`

同时处理表格 Markdown、Office 转换以及可选 OCR。

06 · Docker 化部署

通过 Docker 将服务、依赖与部署环境进行封装，并处理容器访问 Ollama 时的网络拓扑问题。

核心难点

评测公平性

避免将“候选数量变化”和“是否使用 Reranker”混为同一个变量。

Hybrid 截断策略

Dense / BM25 双路召回经过 RRF 后必须稳定截断到 `recall_k`，否则 `candidate_count` 漂移会影响评测和线上行为。

多格式文档处理

不同 Office 文档、表格以及低文本页面在解析过程中存在不同的数据质量问题，需要在入库阶段处理。

本地模型部署

Docker 环境下 Ollama 地址不能简单使用 `127.0.0.1`，需要根据容器拓扑进行配置。

Project 04 | 医疗 AI 细胞分类与模型优化

简要概述

定位：深度学习 / AI 算法 / 医疗图像分析

围绕 巨幼样变细胞分类 开展多模型实验与模型评估。

项目包含：

- 多类别细胞图像分类
- CNN / Transformer 多模型对比
- 模型训练与验证
- 分类性能评估
- 视觉特征与细胞形态特征分析
- 模型解释与错误分析

数据与任务

数据集包含 **8 类巨幼样变相关细胞类别**，采用 **8:1:1** 的训练 / 验证 / 测试划分。

类别包括：

- 早 / 中 / 晚巨幼红细胞
- 巨中 / 巨晚幼粒细胞
- 巨分叶核粒细胞
- 巨杆状核粒细胞
- 巨核细胞巨幼变

模型实验

实验覆盖 CNN 与 Vision Transformer 等不同视觉架构，包括：

- ResNet50
- DenseNet161
- ConvNeXt
- EfficientNetV2
- ViT-S
- DeiT-S
- SwinV2-T
- Nest-T

通过多模型实验比较不同网络结构在细胞分类任务上的表现。

多模态特征分析

进一步结合视觉模型与视觉语言模型，对细胞形态信息进行结构化描述，例如：

- 细胞体积
- 核质比例
- 染色质特征
- 核仁数量
- 胞质染色强度

并结合模型结果进行特征分析、错误分析与可解释性探索。

模型评估

重点关注：

- Accuracy
- Per-class Accuracy
- ROC
- Confusion Matrix
- SHAP / 特征贡献分析

当前实验中保存的模型结果包含 **ResNet50 / Nest-T** 等模型，并进一步分析不同类别之间的混淆情况。

该项目主要体现我在 **深度学习模型训练、实验对比、分类评估和多模态特征分析** 方面的实践经历。

Capability Map

AI 应用开发

1	Python
2	FastAPI
3	LangChain
4	Ollama
5	Docker
6	SQLite
7	Taro
8	Structured Output
9	Pydantic
10	LLM Provider Adapter

能够从需求出发完成：

产品设计 → LLM 接入 → 后端 API → 数据存储 → 前端交互 → Docker 部署

RAG / AI Agent

1	Embedding
2	Dense Retrieval
3	BM25
4	RRF
5	Reranker
6	Query Router
7	Query Rewrite
8	Memory
9	Trace
10	Evaluation

重点关注：

检索质量、上下文构造、Agent / Workflow 边界、可观测性与评测。

数据分析

1	Python
2	SQL
3	Power BI
4	RFM
5	指标体系
6	用户分析
7	经营分析
8	Dashboard

能够将：

业务问题 → **数据处理** → **指标设计** → **可视化** → **分析结论**

形成完整分析链路。

深度学习

1	PyTorch
2	CNN
3	Vision Transformer
4	图像分类
5	模型对比
6	Confusion Matrix
7	ROC
8	SHAP
9	多模态特征分析

具备从 **数据集处理** → **模型训练** → **实验对比** → **性能评估** → **错误分析** 的实践经验。

Engineering Practice

除了具体项目，我持续关注 AI 项目的工程化问题：

Git / GitHub

- Branch / Merge / Rebase
- Conventional Commits
- Pull Request
- Reset / Revert
- 项目 README 与文档沉淀

AI Coding

日常使用 **ChatGPT**、**Cursor**、**Claude Code** 辅助代码编写、调试、重构、文档整理与项目迭代。

我更关注的工作方式是：

```
1  AI 生成
2  ↓
3  人工理解
4  ↓
5  运行验证
6  ↓
7  发现问题
8  ↓
9  修改设计
10 ↓
11 测试
12 ↓
13 沉淀文档
```

而不是将 AI 生成的代码直接视为最终结果。

Appendix | Project Links

- **Enterprise RAG**
<https://github.com/SomeH-Bosx/enterprise-rag>
- **FinPilot**
<https://github.com/SomeH-Bosx/FinPilot>
- **html2img-tool**
<https://someh-bosx.github.io/html2img-tool/>

Contact

HUO BOXS

AI Application · RAG / Agent · Deep Learning · Data Analysis

GitHub: **SomeH-Bosx**